Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Gözden geçirme ve onaylama (review) tekrar edilir: amacı, toplantı rolleri, tipleri, seçenek matrisi, süreç basamaklarına göre neyin gözden geçirildiği.
- Gözden geçirme standartlarına örnekler verilir.
- İnceleme (inspection) işlenir: statik doğrulama olarak yeri, incelenebilen ara ürünler, genel inceleme süreci, Fagan inceleme yöntemi, kod okuma, inceleme ekibi, teknik inceleme özet raporu.
- Doğrulama (verification) ve geçerleme (validation) terimlerinin Türkçe karşılıkları netleştirilir. Statik ve dinamik doğrulama ayrılır.
- V model yaklaşımı ile yazılım geliştirme adımları ve test adımları eşleştirilir.
- Test tanımı ve test süreci adımları: birim testi (unit test), bütünlük testi (integration test), regresyon testi, sistem testi, kabul testi, alfa ve beta testi, teşhis testi.
- Birim testinde test sürücü (test driver) ve koçan (stub) kavramları, test senaryosu (test case) ve birim test araçları anlatılır.
- Derste kısa bir soru-cevap bölümü yapılır. Proje grubu ve yazılım seçimi için son tarih verilir.

Hocanın Özellikle Vurguladığı Kısımlar

- İnceleme, kodu çalıştırmadan hata bulmaya dayanır. Bu yüzden statik doğrulamadır (static verification). Test ise yazılımı çalıştırır; dinamik doğrulama ve geçerlemedir (dynamic verification and validation).
- Dinamik doğrulama yalnızca birim testi değildir. Program parçasını veya bütününü çalıştırarak yapılan bütün testler (birim, bütünlük, sistem) dinamik doğrulamadır.
- Gereksinim spesifikasyonu ve tasarım belgeleri gibi yazılı ürünler çalıştırılarak test edilemez. Bunlar inceleme ile değerlendirilir. Prototip varsa prototip çalıştırılarak test edilebilir.
- Terimler: doğrulama (verification) “ürünü doğru yolla üretiyor muyuz?”, geçerleme (validation) “doğru ürünü mü ürettik?” demektir. Hoca, test alanında çalışanların son kabulüne göre slaytlarını bu şekilde düzelttiğini söyler. Daha önce Türkçede tersi kullanılıyordu.
- Hata varsa onu kenarlarda, yani modül sınırlarında arayın. Sınır değerler mutlaka test edilmelidir.
- Fagan yöntemi çok eskidir, ama ayrıntılı inceleme için bugün de kullanılan süreç tanımıdır.
- Hoca, ekran paylaşımında slayt donarsa öğrencilerin hemen haber vermesini ister.

Kısa Tekrar Notları

- Yazılım hatalarının yaklaşık %65’i tasarım aşamasında yapılır. Gözden geçirmede bunların yaklaşık %75’i bulunup düzeltilebilir. Test öncesinde bulunan hata daha ucuza düzeltilir.
- Gözden geçirme tipleri (en az kurallıdan en kurallıya): formal olmayan gözden geçirme (informal review), yapısal denetim (walkthrough), inceleme (inspection). Round robin peer review incelemenin özel bir halidir.

- İnceleme yapılabilen ürünler: gereksinim spesifikasyonu, yüksek seviye (genel) tasarım, formal spesifikasyon, ayrıntılı tasarım, program kodu.
- Genel inceleme süreci: planlama ve hazırlık; inceleme ve toplantı; düzeltme ve tamamlama.
- Fagan inceleme adımları: planlama (planning), genel bakış (overview), kişisel inceleme / hazırlık (preparation), inceleme toplantısı (inspection meeting), düzeltme (rework), tamamlama ve raporlama (follow-up).
- İnceleme ekibi: inceleme sorumlusu (moderatör), inceleyiciler, kodu yazan, kayıtçı.
- İnceleme sonucu üç karardan biridir: değişiklik yapılmadan kabul; önemsiz hataların düzeltilmesi koşuluyla kabul; red, düzeltme ve yeniden inceleme.
- Test: bir programdaki hataları bulmak amacıyla yapılan işlemlerdir. Yazılımın işlevsel, performans, dayanıklılık ve yapısal açıdan yeterliğini denetler.
- İşlevsel olmayan gereksinimler (performans, dayanıklılık) V modelde sistem testinde test edilir.
- Test sürücü (driver): test edilen modülü çağıran üst modülü temsil eden yapay modül. Koçan (stub): test edilen modülün çağırdığı alt modülleri temsil eden yapay modül.
- Bütünlük testi stratejileri: yukarıdan aşağı (top-down), aşağıdan yukarı (bottom-up), ikisinin birlikte kullanıldığı sandviç (sandwich). En çok sandviç kullanılır.
- Regresyon (tamamlama) testi: bir modülde değişiklik yapıldıktan sonra etkilenen tüm modüllerin ve yazılımın tamamının yeniden test edilmesi.

Detaylı Açıklamalar

Gözden geçirmenin tekrarı

- Gözden geçirme; işlevsel, mantıksal ve gerçekleştirme (kodlama) hatalarının bulunup giderilmesidir.
- Ek faydaları: gereksinimlere uygunluğun onaylanması, standartlara uygunluk, düzenli geliştirme, proje yönetiminin kolaylaşması, müşteriye verilecek belgelerin kalitesinin artması.
- Gözden geçirme ve onun bir kademe ileri hali olan inceleme, kalite sağlama aktivitelerinin en önemlileridir.
- Toplantıdaki roller:
  - Gözden geçirme lideri veya toplantıyı yöneten bir katılımcı.
  - Kullanıcı temsilcisi: ürünün sistem analistleriyle belirlenen gereksinimlere uygunluğunu kontrol eder.
  - Kalite güvence ekibinden biri: kurum standartlarına uygunluğu kontrol eder. Kurumda böyle bir ekip yoksa bu işi standartları belirleyen yazılım ekibi yapar.
  - Kayıtçı.
  - Üretici (yazılım geliştirici ekipten biri): yapılan işi tanımlar, bulunan hataların nasıl giderileceğine dair not alır.
- Küçük projelerde gözden geçirme ve düzeltmeyi geliştirici doğrudan yapabilir. Orta ve büyük projelerde hata ve eksikleri genelde üçüncü bir kişi bulur.

Seçenek matrisinin hocanın anlatımıyla özeti:

	Formal olmayan GG	Walkthrough	Inspection
Gözden geçirme lideri	yok	var	var
Toplantı takvimi	belki (basit)	var	var
Gözden geçirenlerin ön hazırlığı (kodları, raporları önceden inceleme)	belki	var	var
Yazılımcı sunum yapıyor mu	belki	var	yok (doğrudan ürünler incelenir)
Hataları bulmak için checklist	yok	yok	var
Hatalar sınıflanıp sayılıyor mu	yok	yok	var

	Formal olmayan GG	Walkthrough	Inspection
Kayıtçı	belki	var	var

Süreç basamaklarına göre gözden geçirme

Gözden geçirme durak noktalarında (milestone, kilometre taşı) yapılır: sistem analizi, yazılım geliştirme planı, gereksinim analizi, tasarım, kodlama, test, bakım-onarım.

- Sistem analizi: sistem düzeyindeki spesifikasyonların yazılım işlevlerine dağılımının doğruluğu; ana işlevler, ara birimler, performans ve tasarım kısıtları; seçimlerin tutarlılığı; teknik yapılabilirlik (feasibility); test esasları; sistem öğeleri arasındaki uyum.
- Yazılım geliştirme planı: risk, sorun tanımı, terimler, kaynakların yeterliliği, görevlerin uygunluğu ve sırası; verimlilik, kalite, gider ve zamanlama tahminleri; seçilen kişilerin uygunluğu.
- Gereksinim analizi: gereksinimlerin tutarlılığı ve gerçek isteklere bağlılığı; zaman, kaynak ve bütçe açısından yapılabilirliği; onaylama kriterlerinin doğru konması.
- Tasarım:
  - Ön tasarım: gereksinimlerin tasarıma ve yazılım mimarisine doğru aktarılması, modüllerin etkinliği, modüller arası işlevsel bağımsızlık, program yapısının uygunluğu.
  - Ayrıntılı tasarım: veri yapıları ve işlemlerin doğruluğu.
- Kodlama (gerçekleme): ayrıntılı tasarımın programlama diline doğru dönüştürülmesi, yazılım hataları, kullanılan dilin kodlama standartlarına uygunluk, açıklamaların (yorumların) tutarlılığı, tasarım hatalarının düzeltilip düzeltilmediği. Kodlama aşamasındaki gözden geçirme çoğunlukla review yerine kod incelemesi (code inspection) ile yapılır.

Gözden geçirme standartlarına örnekler: sistem tasarımı gözden geçirmesi (system design review), yazılım spesifikasyonu gözden geçirmesi (software specification review), ön tasarım gözden geçirmesi (preliminary design review), teste hazır olmanın değerlendirilmesi (test readiness review).

İnceleme (inspection)

- İnceleme, testle birlikte yazılım kalitesini sağlamak için en çok kullanılan yöntemlerden biridir.
- Testin aksine hataları kodu çalıştırmadan bulur. Bu yüzden statik doğrulamadır. "Doğrulama"dır, çünkü gidiş yolunun doğruluğuna bakar: gereksinim spesifikasyonunu, tasarımı, formal spesifikasyonu doğru hazırladık mı?
- Bütün yarı ürünler üzerinde inceleme yapılabilir. Gereksinim spesifikasyonu incelenirken bağlantılı prototip de incelenebilir.

```
Statik doğrulama (inceleme):
  Gereksinim spesifikasyonu, yüksek seviye tasarım, formal spesifikasyon,
  ayrıntılı tasarım, program

Dinamik doğrulama ve geçerleme (test):
  Prototip, program
```

Genel inceleme süreci

1. Planlama ve hazırlık: incelemenin amacı ve hedefleri, incelenecek yazılım parçaları, incelemeyi kimin yapacağı ve bu kişilerin özellikleri, uygulanacak inceleme teknikleri ve takip eylemleri belirlenir. Gerekli altyapı sağlanır.
2. İnceleme ve toplantı: herkes kendine düşen kodu veya rapor bölümünü belirlenen kriterlere göre inceler. Bulunan hatalar düzeltilmek üzere kayda alınır. Sonra ekip bir araya gelir.
3. Düzeltme ve tamamlama: hatalar sorumlu kişilerce düzeltilir ve takip edilir. İstenen sonuç elde edilemezse sürecin başına dönülür ve yeniden inceleme yapılır.

Fagan incelemesi Fagan yöntemi genel süreci daha fazla alt adıma böler:

1. Planlama.
2. Planın gözden geçirilmesi: görevler ve herkese verilen kod bölümleri gözden geçirilir. Plan doğru mu kontrol edilir.
3. Kişisel kod inceleme: herkes kendine verilen bölümü inceler.
4. İnceleme toplantısı.
5. Düzeltme: toplantıda ortaya çıkanlar yazılım ekibine düzeltme için gönderilir.
6. Tamamlama ve raporlama.

Fagan incelemesini diğer incelemelerden ayıran özellikler: raporlama ve toplantı biçiminin kurallı olması, ön hazırlık yapılması, kimin neyi inceleyeceğine ekip yapısı içinde karar verilmesi.

Etkili kod okuma:

- Yukarıdan aşağıya (top-down) ve aşağıdan yukarıya (bottom-up) incelemenin birleşimi olmalıdır.
- En genel program modülünden başlayıp çağırdığı modüllere inilir. Alttaki ayrıntı modülleri de ayrıntılı incelenir.
- Program mümkün olduğunca küçük parçalara (prosedür, alt program) bölünür. Büyük resim kaybedilmeden her küçük parçanın doğruluğu denetlenir.

İnceleme ekibi: inceleme sorumlusu, inceleyiciler (kod okuma ve incelemeyi yapanlar), kodu yazan, rapor yazan / kayıt tutan.

Teknik inceleme özet raporunda bulunması gerekenler:

- Neyin gözden geçirildiği.
- Kimin gözden geçirdiği (alt modüllerin kime verildiği eşleştirilerek).
- Hangi hata ve eksiklerin bulunduğu ve ne yapılması gerektiği.
- Sonuç kararı: değişiklik yapılmadan kabul; önemsiz hataların düzeltilmesi koşuluyla kabul; çok sayıda önemli hata nedeniyle red, düzeltme sonunda yeniden inceleme.

V model yaklaşımı

Model hem şekli nedeniyle hem de verification ve validation sözcüklerinin baş harfi nedeniyle "V" adını alır. Sol kol yazılım geliştirme aktiviteleridir, yukarıdan aşağı ilerler. Sağ kol test aktiviteleridir, aşağıdan yukarı ilerler. Her geliştirme adımı karşısındaki test adımıyla eşleşir.

```
Müşteri gereksinimleri  ---- Geçerleme -------------> İşlemsel kullanım (+ Beta testi, Teşhis tes
  Ürün spesifikasyonu   ---- Geçerleme/Doğrulama --->  Sistem testi (+ Kabul testi?)
   Yüksek düzey tasarım ---- Doğrulama -----------> Bütünlük testi (+ Regresyon testi)
      Alçak düzey tasarım ---- Doğrulama --------->  Bileşen testi
               Kodlama ve birim testi (V'nin dibi)
```

- Müşteri gereksinimleri: müşterinin hangi problem için yazılım istediği ve beklentileri, müşterinin ağzından madde madde.
- Ürün spesifikasyonu: analistin geliştiricilerle birlikte işlevsel ve işlevsel olmayan gereksinimlerle yaptığı formal tanım.
- Yüksek düzey (genel, mimari) tasarım: modül hiyerarşisi, yapı diyagramları (structure diagram). Bütünlük testinde bu yapı kontrol edilir.
- Alçak düzey tasarım: her modülün iç yapısı, algoritması, arayüzleri, veri tabanı tasarımı. Birim ve bileşen testinde kontrol edilir.
- Sistem testinde gereksinim spesifikasyonu kullanılır. Hem "işi doğru yürüttük mü?" hem "doğru ürünü elde ettik mi?" sorulur. Bu yüzden hem doğrulama hem geçerleme yapılır.
- İşlemsel kullanımda müşterinin istekleri (iş analistinin aktardığı haliyle) kullanılır. Yalnızca geçerleme yapılır.
- Alt basamaklarda yalnızca doğrulama yapılır.
- Kabul testinin şemadaki yeri büyük olasılıkla bu şekildedir; hoca kabul testini işlemsel kullanımla birlikte de andı (01:57:36).
- V model şelale modeline göre tanımlanmıştır: önce geliştirme bitirilir, sonra test adımları aşağıdan yukarı yapılır.
- Hoca, eski yazılım mühendisliği kitabındaki şekil yerine bu V model şeklini akılda tutmayı önerir.

Test süreci adımları

Birim testlerini geliştiriciler yapar. Bütünlük ve sistem testini test grubu yapar. İşlemsel kullanım testinin bir kısmını test grubu, bir kısmını potansiyel kullanıcılar yapar.

Birim testi

- Yazılım tasarımının en küçük birimi olan modül üzerinde uygulanır. Ayrıntılı tasarım tanımlarına (prosedür tasarımı, veri tabanı tasarımı, arayüz tasarımı) dayanılarak modülün içi değerlendirilir.
- Genellikle saydam kutu testi (white box test) uygulanır: yazılım biriminin iç yapısı incelenir. Hoca "white box" için "saydam kutu" terimini kullanır, çünkü içi görünür.
- Geliştiriciler kendi yazdıkları modüllerin testini test ekibine gitmeden yapar. Algoritmaları ve kontrol yollarını bildikleri için test verisini tüm kontrol yollarını deneyecek şekilde seçebilirler. Bu yüzden birim testleri çok sayıda modül üzerinde paralel yürür.
- Birim testinde sınanan noktalar: modülün ara birimleri (interface), yerel veri yapıları, kontrol yapılarındaki ana yollar, hata işleme yolları, modül sınırları.

Test sürücü ve koçan düzeni:

```
      Test sürücü (driver)
   test verisini gönderir, sonucu dışarı aktarır
            |
      Test edilen birim
      /       |       \
Koçan 1   Koçan 2   Koçan 3
(stub: çağrılan alt modüllerin yerine geçen yapay modüller)
```

- Test sürücü: test verisi modüle doğrudan gönderilemiyorsa (modülü çağıran bir üst modül varsa) o üst modülü temsil eden yapay modüldür.
- Koçan (stub): test edilen modülün çağırdığı, henüz hazır veya test edilmemiş alt modüllerin yerine geçer. Veri tabanı bağlantılarının yerine de konabilir.
- Modül bağımsız değilse her testte sürücü ve/veya koçan yazılımı gerekir.
- Basit durumda amaç: test senaryolarındaki girdileri birime ver, sonuçları al, elle hesaplanan sonuçlarla karşılaştır.
- Gerçek hayatta test senaryosu oluşturma, test verisi seçme, çalıştırma ve yapay alt programları oluşturma için araçlar kullanılır. Örnek: Java için JUnit.

Test senaryosu (test case):

- Belirli bir program yolunu işletmek veya belirli bir gereksinime uygunluğu onaylamak için düzenlenen test verileri ve ilgili işlemlerdir.
- Saydam kutu testinde test verisi bir kontrol akışı yolunu denemek için seçilir.
- Dışarıdan (kullanıcı gözüyle) testte test verisi kullanıcının belirlediği bir gereksinimi sınamak için seçilir.
- Test programları da diğer yazılımlar gibi geliştirilir, hazır da alınabilir.

Bütünlük testi

- Test edilmiş modüller birleştirilerek test edilir. Tüm modüller test edildikten sonra veya bir kısmı birleştirilerek yapılabilir.
- Stratejiler: yukarıdan aşağı, aşağıdan yukarı ve sandviç. En çok sandviç kullanılır.
- Ayrıntıları bir sonraki derste işlenecektir.

Regresyon (tamamlama) testi

- Bütünlük testinde birleştirilmiş modüllerden birinde değişiklik veya düzeltme yapılırsa, bu değişikliğin etkilediği tüm modüller ve yazılımın tamamı yeniden test edilir.
- Birim testinde olduğu gibi regresyon testinde de CASE araçları ve test araçları kullanılır.

Kabul testi, alfa testi, beta testi, teşhis testi

- Kabul (onaylama) testi (acceptance test): teslim sırasında kullanıcının yaptığı testtir.

- Alfa testi: sistem testinden sonra, yazılım evinde kullanıcının yerine geçen bir grubun yazılımı kullanıcı gözüyle test etmesidir.
- Beta testi: kullanıcı kabulünden sonra yazılım potansiyel kullanıcılara dağıtılır; onların hata bulması ve kullanıma geçmesi istenir.
- Teşhis testi: işlemsel kullanım sırasında hata çıkarsa yapılan testtir.

Soru-cevap bölümü

- İnceleme ile gözden geçirme farkı: inceleme daha ayrıntılıdır, kontrol listesi ve hata sınıflaması içerir, ön hazırlık ve kayıt kurallıdır.
- Statik doğrulama hangisidir? İnceleme (program çalıştırılmaz).
- Dinamik doğrulama hangisidir? Test. Bir öğrenci birim testi der. Hoca birim testinin doğru ama eksik olduğunu, bütünlük ve sistem testinin de dinamik doğrulama olduğunu ekler.
- Koçan (stub) nedir? Çağrılan alt programların yerini tutan yapay modüllerdir.
- Test senaryosu nedir? Bir kodu veya bir gereksinimi sınamak için uygun test verilerinin oluşturulmasıdır. Kontrol akışından çıkan yollar, veri grupları veya kullanıcının istediği özellikler olabilir.

Proje duyurusu

- Bu hafta hiçbir gruptan bilgi gelmemiştir. Bir sonraki derse (4. hafta, aynı gün) kadar e-posta atılmalıdır.
- E-postada istenenler: 5 kişilik grubun üyeleri ve test edilecek yazılım hakkında 2-3 cümle (adı, konusu, nereden geldiği, hangi dilde yazıldığı). Görev dağılımı şimdilik istenmez.
- Grup listesi verildikten sonra yeni üye eklenemez.
- Hoca konu vermez. Yazılımı grup bulur. Bir üyenin ara projesi, başka bir dersin projesi, bir kütüphane veya herhangi bir yazılım olabilir. Kaynak koduna erişilebilmelidir, çünkü bir birim testi kod üzerinden yapılacaktır.
- Büyük bir yazılım seçilirse belli bir parçası üzerinde çalışılabilir.
- Testler: birim testi, sistem testi ve gereksinim listesine dayalı kullanıcı testi.
- Gereksinim listesi 50-70 madde yeterlidir. Her gereksinim ayrı bir madde olarak yazılır. İşlevsel gereksinimler modüllerin yaptığı işler sınıflandırılarak çıkarılır. İşlevsel olmayan gereksinimler kalite özellikleridir. Gereksinim listesinin ne olduğuna yazılım mühendisliği kitabından veya ders notlarından bakın.

Ders sonrası notlar

- Soru-cevapta bir öğrenci “incelemede yazılımcı sunum hazırlar” der. Seçenek matrisine göre yazılımcının sunum yaptığı tip walkthrough’dur; inspection’da yazılımcı sunum yapmaz.
- Aynı öğrenci “incelemede test ederek hata bulunur” der. Bu yanlıştır: inceleme kodu çalıştırmadan yapılan statik doğrulamadır. Çalıştırarak hata bulmak testtir.
- Bir başka öğrenci “incelemede çözüm yolları aranır” der. İncelemenin amacı hataları bulmak, sınıflamak ve kaydetmektir. Düzeltme ayrı bir adımda (rework) geliştiriciye bırakılır.
- Fagan’ın özgün tanımında ikinci adım (overview), incelenecek ürünün ekibe kısaca tanıtıldığı bir toplantıdır. Hoca bu adımı “planın ve görev paylaşımının gözden geçirilmesi” olarak anlatır.
- Hoca başta hataların “%50’si, 65’i” tasarımda yapılır diye düzeltir. Slayttaki değer %65’tir.
- Whisper “Fagan”ı bir yerde “Şafozy” diye, “stub” sözcüğünü “Stop/Stab”, “koçan”ı “Koçan”, “bilim testi”ni birim testi yerine yazmıştır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.